

최신 주식 정보를 알려 주는 AI 투자자

GPT 언어 모델은 똑똑하지만 ‘지금 몇 시지?’, ‘오늘 비 오니?’ 같은 간단한 질문에 답하지 못합니다. API를 통해 사용하는 GPT 모델은 마치 시계도 창문도 없는 감옥에 갇혀 있는 사람처럼 제한된 환경에서 작동하기 때문입니다. GPT의 이러한 한계를 보완하기 위해 **평선 콜링**이라는 기술이 있습니다. 이번 장에서는 파이썬 함수를 활용해 GPT가 더 다양한 작업을 할 수 있게 하는 방법을 알아보겠습니다.

하네스의 시작

07-1 평선 콜링의 기초

07-2 GPT와 미국 주식 이야기하기

07-3 스트림 출력하기

실시간 출력

07-1 평선 콜링의 기초

GPT야, 지금 몇 시지?

03-2절 'GPT와 멀티턴 대화하기'에서 만든 챗봇(`multi_turn.py`)에 '지금 몇 시지?'라고 물어 보면 다음처럼 알려 줄 수 없다는 답변이 나옵니다.

사용자: 지금 몇 시지?

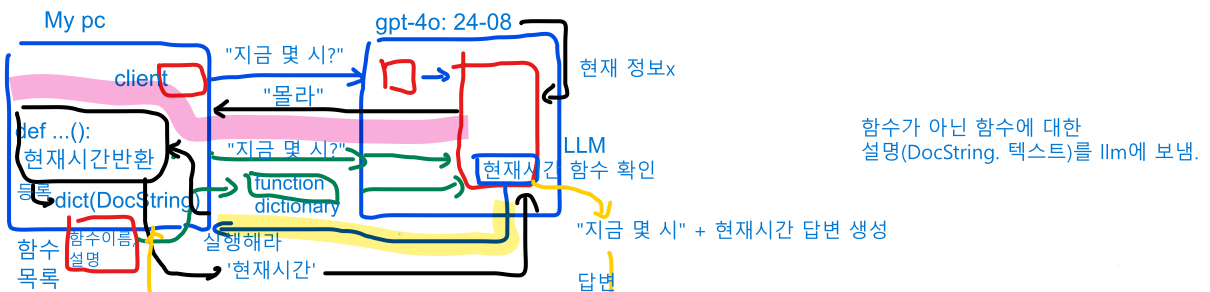
AI: 죄송하지만 현재 시간을 알려드릴 수는 없습니다. 사용 중인 기기의 시계를 확인해보시기 바랍니다.

GPT 같은 언어 모델은 아무리 똑똑해 보이더라도 기본적으로 이전 텍스트를 기반으로 다음에 나올 문장을 예측하는 모델입니다. 학습을 종료한 시점에 잠들어 있다가 API를 통해 호출될 때마다 깨어나 답변을 해주고 다시 잠드는 사람과 비슷합니다. 따라서 현재 시간이 몇 시인지, 밖의 날씨가 어떤지, 최근에 유행하는 음악은 무엇인지 알지 못합니다. GPT와 대화한 내용을 파일로 저장해 달라고 요청해도 그런 작업은 수행할 수 없습니다.

평선 콜링이란? 현재를 알 수 있게 해줌.

이러한 GPT의 한계를 극복하기 위해 오픈AI에서는 평선 콜링(Function calling) 기능을 제공합니다. 평선 콜링을 활용하면 GPT에게 함수와 그에 관한 설명을 제공하고 상황에 맞는 특정 함수를 호출하도록 할 수 있습니다. GPT는 함수 실행 결과를 해석하여 사용자에게 답변을 제공합니다.

오픈AI에서는 평선 콜링 기능에 사용할 수 있는 함수를 담은 도구 목록을 딕셔너리 형태로 정의합니다. 예를 들어 사용자가 '지금 몇 시야?'라고 물으면 GPT는 이 도구 목록에서 시간을 확인할 수 있는 도구가 있는지 찾아서 그 도구를 사용해 답변합니다. 즉, 도구 목록의 딕셔너리는 GPT 모델이 어떤 도구를 사용할 수 있는지 알려 주는 설명서 역할을 하며, GPT API를 호출할 때 이 도구 목록도 함께 전달됩니다.

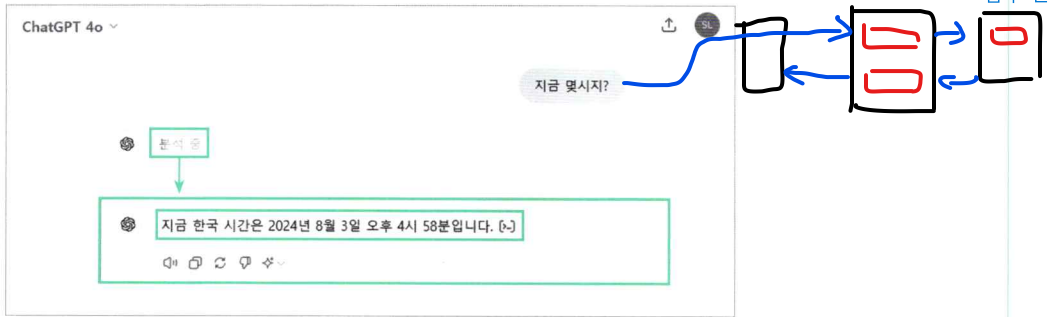


★ 한 걸음 더!

챗GPT는 지금 몇 시인지 잘 대답하나요?

챗GPT는 GPT-4o 모델을 기반으로 이 장에서 배울 평션 콜링을 비롯한 여러 기능이 추가된 서비스입니다. 사용자가 챗GPT에 '지금 몇 시지?' 라고 질문하면 '분석 중...' 이라는 메시지가 깜박이다가 최종 답변이 나옵니다. 이 메시지는 평션 콜링을 사용해 정보를 생성하는 과정을 나타냅니다.

function calling
openai가
미리 자주 쓰이는
함수 만들어놓음.



평션 콜링 기능을 사용해 질문에 답변하는 챗GPT

Do it! 실습

평션 콜링 적용하기

function
tools
리스트
(부품)
모듈

사용

■ 결과 파일: chap07/sec01/gpt_functions_0.py, sec01/what_time_is_it_terminal_0.py

스트림릿으로 만든 챗봇에 평션 콜링을 적용해 지금이 몇 시인지 GPT가 답할 수 있도록 구현해 보겠습니다.

1. 파이썬 파일 `gpt_functions.py`을 생성합니다. 여기서 평션 콜링에 사용할 함수를 정의하고 뒤에서 만들 코드에 이 파일을 `import`하여 사용할 계획입니다. GPT가 현재 시간을 파악할 수 있게 해주는 `get_current_time` 함수를 작성합니다. 이 함수는 매개변수는 사용하지 않으며 실행 여부를 터미널 창에서 확인하기 위해 현재 시간을 출력하고 반환합니다.

한국 기준

GPT에서 사용할 함수 정의하기

서울

gpt_functions.py

```
from datetime import datetime (레퍼런스 변수)
    모듈
def get_current_time():
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(now)
    return now

if __name__ == '__main__':
    get_current_time()
```

현재 날짜/시간 반환 메소드 -> 숫자
메소드

이 코드를 실행하면 결과로 현재 시간을 터미널 창에 출력합니다.

2025-01-21 16:35:58

2. 현재 시간을 알려 주는 함수를 평션 콜링 기능으로 활용하기 위한 설명을 추가하겠습니다.

GPT를 위해 사용할 함수 설명 추가하기

gpt_functions.py

```
from datetime import datetime

def get_current_time():
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(now)
    return now

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_current_time",
            "description": "현재 날짜와 시간을 반환합니다.",
        }
    },
]

if __name__ == '__main__':
    get_current_time()
```

- 1 새로 만든 파이썬 함수에 대한 설명은 tools라는 리스트에 딕셔너리 형태로 담아 두었습니다. 현재 tools에 get_current_time 함수만 포함되어 있지만 필요에 따라 딕셔너리 형태로 추가해 나가면 됩니다.
- 2 name에는 함수의 이름, description에는 이 함수를 언제 사용할 수 있는지에 대한 설명을 기록합니다. 일반적으로 name과 description 이외에도 함수에 필요한 매개변수를 parameters에 적어야 하지만 이번 함수는 매개변수를 사용하지 않으므로 추가하지 않았습니다.

이제 03-2절에서 작성한 코드(multi_turn.py)를 수정해서 터미널 창에서 평션 콜링 기능을 사용하는 방법을 익히겠습니다. 코드를 조금씩 수정하며 GPT의 응답을 확인해 봅시다.

3. 새 파일 `what_time_is_it_terminal.py`를 만듭니다. `multi_turn.py` 코드를 붙여넣고 평선 콜링에 사용할 도구 목록인 `tools`를 포함하도록 수정합니다.

```

GPT에 tools 정보 포함하기 ■ what_time_is_it_terminal.py

from gpt_functions import get_current_time, tools—❶
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기

client = OpenAI(api_key=api_key) # 오픈AI 클라이언트의 인스턴스 생성

def get_ai_response(messages, tools=None):—❷
    response = client.chat.completions.create(
        model="gpt-4o", # 응답 생성에 사용할 모델 지정
        messages=messages, # 대화 기록을 입력으로 전달
        tools=tools,—❸
    )
    return response—❹

```

- ❶ 앞서 만든 `gpt_functions.py` 파일의 `get_current_time` 함수와 `tools`를 임포트합니다.
- ❷ `get_ai_response` 함수의 매개변수에 `tools`를 `None`으로 추가합니다. 이 `tools`를 `client.chat.completions.create`에서 사용합니다.
- ❸ `get_ai_response` 함수에서 `tools`에 `gpt_functions`의 `tools`를 대입해 주면 `get_current_time` 함수를 사용할 수 있습니다.
- ❹ 원래 `multi_turn.py`에서는 `response.choices[0].message.content`로 메시지만 문자열 형태로 반환하는 형태였습니다. 여기에서는 생성된 응답을 온전히 반환하도록 수정합니다.

4. 터미널 창에서 반복적으로 호출하기 위해 `get_ai_response` 함수를 `while True:` 내부에서 사용합니다. 사용자가 입력할 때마다 이 반복문 안에서 GPT의 응답을 처리합니다.

```

get_ai_response 함수 실행 준비하기 ■ what_time_is_it_terminal.py

(... 생략 ...)
messages = [
    {"role": "system", "content": "너는 사용자를 도와주는 상담사야."}, # 초기 시스템 메시지
]

while True:
    user_input = input("사용자>t: ") # 사용자 입력받기

```

```

if user_input == "exit": # 사용자가 대화를 종료하려는지 확인
    break

messages.append({"role": "user", "content": user_input}) # 사용자 메시지를 대화 기록에
추가

ai_response = get_ai_response(messages, tools=tools) — ❶
ai_message = ai_response.choices[0].message — ❷
print(ai_message) — ❸

tool_calls = ai_message.tool_calls
if tool_calls: # tool_calls가 있는 경우
    tool_name = tool_calls[0].function.name
    tool_call_id = tool_calls[0].id
    } — ❹

    if tool_name == "get_current_time":
        messages.append({
            "role": "function", # role을 "function"으로 설정
            "tool_call_id": tool_call_id,
            "name": tool_name,
            "content": get_current_time(), # 함수 실행 결과를 content로 설정
        })
    } — ❺

ai_response = get_ai_response(messages, tools=tools) — ❻
ai_message = ai_response.choices[0].message

messages.append(ai_message)
print("AI\t: " + ai_message.content)
} — ❼

```

- ❶ gpt_functions에서 임포트한 tools를 같이 지정합니다. 이제 get_current_time과 같은 외부 함수를 GPT가 선택해 사용할 수 있도록 설정됩니다.
- ❷ get_ai_response 함수의 결과가 ai_response에 저장됩니다. 이전에는 문자열 형태로 응답을 받았지만 이제는 객체 형태로 반환됩니다.
- ❸ ai_message의 값이 어떤 형태인지 확인하기 위해 임시로 출력합니다. 이렇게 하면 AI 응답을 제대로 처리할 수 있는지 확인할 수 있습니다.
- ❹ 만약 GPT가 특정 함수를 실행해야 한다고 판단하면 ai_message의 tool_calls라는 속성에 실행할 함수의 정보가 포함됩니다. tool_calls가 있다면 함수를 실행해야 한다고 GPT가 판단했다는 뜻입니다. 여기서 실행할 함수명을 가져오고 평션 콜링의 id를 받아 옵니다.
- ❺ 만약 이 tool_name이 get_current_time이라면 새로운 메시지의 role을 'function'으로 설정하고 tool_call_id와 tool_name을 포함하여 get_current_time 함수의 실행 결과를 messages에 추가합니다.

6 그 후 `get_ai_response` 함수를 다시 한번 호출하여 GPT의 답을 받습니다.

7 그 결과를 `messages`에 추가하고 터미널 창에 출력합니다.

코드를 실행하고 질문하면 다음처럼 사용자가 요청하는 사항에 따라 잘 답변해 줍니다. 함수를 사용해야 하는 상황에서는 `ChatCompletionMessageToolCall`의 인스턴스로 반환되고 있습니다.

사용자: 안녕?

`ChatCompletionMessage(content='안녕하세요! 어떻게 도와드릴까요?', refusal=None, role='assistant', audio=None, function_call=None, tool_calls=None)` — `ai_message`에서 출력된 내용

AI: 안녕하세요! 어떻게 도와드릴까요?

사용자 : 지금 몇시야?

`ChatCompletionMessage(content=None, refusal=None, role='assistant', audio=None, function_call=None,` — `ai_message`에서 출력된 내용

`tool_calls=[ChatCompletionMessageToolCall(id='call_Sa5GVNORrKe0pAMq2vkdSEUH', function=Function(arguments='{}', name='get_current_time'), type='function'))]`

2025-01-06 18:49:55

`get_current_time` 함수에서 출력된 내용

AI: 현재 시간은 2025년 1월 6일 오후 6시 49분입니다.

뉴욕은 지금 몇 시야?

GPT는 제가 어디에 있는지 알 수 없습니다. 평선 콜링 기능을 이용해 GPT가 시간을 알려 줄 수 있지만 이것이 런던 시간인지 서울 시간인지 알지 못합니다. 저는 지금 서울에 있고 현재 시각은 2025년 1월 6일 19시 9분입니다. GPT에게 다짜고짜 지금 뉴욕은 몇 시냐고 물어보면 잘못된 답변을 합니다. `get_current_time` 함수를 통해 얻은 시각이 그리니치 표준시인 UTC 기준이라고 판단해 그 시간을 기준으로 뉴욕이 몇 시인지 계산하기 때문입니다. 하지만 19시 9분은 그리니치 표준시가 아닌 서울의 시간이므로 맞지 않는 답변입니다.

사용자: 지금 뉴욕은 몇 시야?

2025-01-06 19:09:24

AI: 현재 UTC 시간은 2025년 1월 6일 19시 9분 24초 입니다. 뉴욕은 동부 표준시(EST)를 사용하므로, UTC 시간보다 5시간 늦습니다. 따라서 뉴욕의 현재 시간은 2025년 1월 6일 14시 9분 24초입니다.

이 문제를 해결해 보겠습니다. 파이썬에서는 지역별로 다른 시간대(타임존)에 맞는 시간 정보를 얻기 위해 `pytz` 라이브러리를 제공합니다. 예를 들어 'Asia/Seoul', 'America/New_York'과 같은 형식으로 타임존 정보를 입력하면 지역별 시간을 활용할 수 있습니다.

Do it! 실습 도시별 시간 알려 주기

■ 결과 파일: sec01/gpt_functions.py, sec01/what_time_is_it_terminal_1.py

pytz를 활용하여 GPT가 특정 도시의 시간을 알려 줄 수 있도록 구현해 보겠습니다.

1. 앞에서 만든 gpt_functions.py 파일을 수정합니다. 해당되는 타임존의 시각을 반환하기 위해 get_current_time 함수가 타임존 매개변수를 받도록 합니다. pytz을 사용해 문자열 형식으로 받은 타임존을 파이썬 타임존 인스턴스로 만들고 이를 datetime.now에 넣어 해당 타임존의 시각을 구합니다. 실행한 결과가 어느 지역인지 알 수 있도록 반환되는 값에 타임존 정보를 추가합니다.

타임존 정보를 이용해 현재 시간을 구할 수 있도록 수정하기(1)

■ gpt_functions.py

```
from datetime import datetime
import pytz

def get_current_time(timezone: str = 'Asia/Seoul'):
    tz = pytz.timezone(timezone) # 타임존 설정
    now = datetime.now(tz).strftime("%Y-%m-%d %H:%M:%S")
    now_timezone = f'{now} {timezone}'
    print(now_timezone)
    return now_timezone
(... 생략 ...)
```

2. 다음으로 GPT가 사용할 수 있는 도구를 담은 tools 리스트에서 get_current_time 함수의 항목을 수정합니다.

타임존 정보를 이용해 현재 시간을 구할 수 있도록 수정하기(2)

■ gpt_functions.py

```
(... 생략 ...)
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_current_time",
            "description": "해당 타임존의 날짜와 시간을 반환합니다.",
            "parameters": {
                "type": "object",
                "properties": {
                    "timezone": {
                        "type": "string",

```

```

        'description': '현재 날짜와 시간을 반환할 타임존을 입력하세요.
(예: Asia/Seoul)',
    },
    },
    "required": ['timezone'],
},
}
},
]

if __name__ == '__main__':
    get_current_time('America/New_York')

```

- ❶ 이전에는 값이 없었던 properties에 timezone을 추가하고 이 항목의 용도를 설명하는 description도 넣습니다. 이 형식은 GPT의 평선 콜링에서 요구하는 구조입니다.
- ❷ 이 함수를 실행하려면 반드시 timezone 파라미터가 있어야 한다고 명시했습니다.
- ❸ 테스트 삼아 'America/New_York'으로 미국 뉴욕의 시각을 출력하는 코드를 입력했습니다.

이렇게 GPT에 도구 목록을 넘기면 이 정보에 기반해 함수 사용에 필요한 변수를 판단하고 GPT가 해당 매개변수를 유추하여 채워 넣거나 사용자에게 다시 물어볼 수도 있습니다.

3. 이제 what_time_is_it_terminal.py에도 앞서 추가한 타임존 정보를 반영합니다.

타임존을 사용하도록 수정하기

what_time_is_it_terminal.py

```

from gpt_functions import get_current_time, tools
from openai import OpenAI
from dotenv import load_dotenv
import os
import json # GPT가 JSON 형태의 문자열을 반환할 때 읽기 위한 라이브러리 임포트

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기

client = OpenAI(api_key=api_key) # OpenAI 클라이언트의 인스턴스 생성하기
(... 생략 ...)

while True:
    (... 생략 ...)

    tool_calls = ai_message.tool_calls # AI 응답에 포함된 tool_calls 가져오기
    if tool_calls: # tool_calls가 있는 경우

```

```

tool_name = tool_calls[0].function.name # 실행해야 한다고 판단한 함수명 받기
tool_call_id = tool_calls[0].id # 함수 아이디 받기

arguments = json.loads(tool_calls[0].function.arguments)—❶

if tool_name == "get_current_time": # tool_name이 "get_current_time"인 경우
    messages.append({
        "role": "function", # role을 "function"으로 설정
        "tool_call_id": tool_call_id,
        "name": tool_name,
        "content": get_current_time(timezone=arguments['timezone']),—❷
    })
(... 생략 ...)
```

- ❶ GPT는 get_current_time 함수를 실행할 때 타임존 정보가 필요하므로 필요한 값을 arguments에 추가합니다. tool_calls[0].function.arguments로 이 값을 채워 줍니다. 이 값을 사용하려면 GPT가 반환한 JSON 형태의 문자열을 딕셔너리로 바꿔야 하므로 json.loads를 사용해 딕셔너리 형태로 바꿔 줍니다. 이를 위해 코드 상단에서 json 라이브러리를 임포트 했습니다.
- ❷ 이렇게 변환된 값은 get_current_time 함수의 인자로 들어갑니다.

이 코드를 실행하면 GPT가 어느 지역 시각을 알고 싶은지 물어보고 그에 맞춰 현재 시간을 알려 줍니다.

```

사용자: 지금 몇시야?
AI: 어느 지역의 시간을 알고 싶으신가요? 특정 타임존이 있으시면 말씀해 주세요.
사용자: 서울
2025-01-06 20:20:01 Asia/Seoul
AI: 서울의 현재 시간은 2025년 1월 6일 오후 8시 20분입니다.
사용자: 런던은?
2025-01-06 11:20:07 Europe/London
AI: 런던의 현재 시간은 2025년 1월 6일 오전 11시 20분입니다.
사용자: exit
```

Do it! 실습 여러 도시의 시간을 한 번에 대답할 수 있게 하기

■ 결과 파일: sec01/what_time_is_it_terminal.py

앞선 실습에서 만든 챗봇에 한 번에 여러 도시의 시간을 물어보면 명확한 답을 주지 못하는 경우가 많습니다. 여러 도시를 물어보니 사용자에게 스스로 계산하라고 하네요. 운이 좋다면 GPT가 언어 능력을 활용해 계산해 줄 수도 있지만 이런 불확실성을 없애려면 함수를 여러 번 호출할 수 있도록 해줘야 합니다.

사용자: 도쿄, 서울, 뉴욕 몇시야?

2025-01-06 20:22:56 Asia/Tokyo

AI: 현재 도쿄의 시간은 2025년 1월 6일 오후 8시 22분입니다. 각 도시의 시간대는 다음과 같으니, 이를 사용하여 서울과 뉴욕의 현재 시간을 계산해 주세요:

- 도쿄 (JST): GMT+9
- 서울 (KST): GMT+9
- 뉴욕 (EST): GMT-5

다음과 같이 GPT가 여러 함수를 한 번에 실행하려고 계획을 세웠을 때 코드에서는 함수 호출을 한 번만 할 수 있게 되어 있어서 오류가 발생할 수도 있습니다.

사용자: 런던, 파리, 베를린 시간 알려줘.

ChatCompletionMessage(content=None, refusal=None, role='assistant', audio=None, function_call=None, tool_calls=[

ChatCompletionMessageToolCall(id='call_a8Y9RMpnnHGjZNE3hS3o3MqF', function=Function(arguments='{"timezone": "Europe/London"}', name='get_current_time'), type='function'),

ChatCompletionMessageToolCall(id='call_hfnxha7I2tlQ8F0qgJtqANNN', function=Function(arguments='{"timezone": "Europe/Paris"}', name='get_current_time'), type='function'),

ChatCompletionMessageToolCall(id='call_MRutImjENKEteWfVDQzLlw40', function=Function(arguments='{"timezone": "Europe/Berlin"}', name='get_current_time'), type='function'))]

2025-02-16 16:03:45 Europe/London

Traceback (most recent call last):

File "c:\github\gpt_agent_2025_easyspub\chap07\sec01\what_time_is_it_terminal.py", line 57, in <module>

print("AI\t: " + ai_message.content) # AI 응답 출력

TypeError: can only concatenate str (not "NoneType") to str

여러 함수가 연속으로 실행되게 하기 위해 GPT가 함수를 차례로 실행할 수 있도록 코드를 수정합니다.

여러 함수가 연속으로 실행되도록 하기 what_time_is_it_terminal.py

```
(... 생략 ...)
tool_calls = ai_message.tool_calls # AI 응답에 포함된 tool_calls 가져오기
if tool_calls: # tool_calls가 있는 경우
    for tool_call in tool_calls: ❶
        tool_name = tool_call.function.name # 실행해야 한다고 판단한 함수명 받기
        tool_call_id = tool_call.id # 함수 아이디 받기
        arguments = json.loads(tool_call.function.arguments) # 문자열을 딕셔너리로 변환

        if tool_name == "get_current_time": # tool_name이 "get_current_time"인 경우
            messages.append({
                "role": "function", # role을 "function"으로 설정
                "tool_call_id": tool_call_id,
                "name": tool_name,
                "content": get_current_time(timezone=arguments['timezone']),
            })
            messages.append({"role": "system", "content": "이제 주어진 결과를 바탕으로
            답변할 차례다."}) ❸

        ai_response = get_ai_response(messages, tools=tools) # 다시 GPT 응답 받기
        ai_message = ai_response.choices[0].message
(... 생략 ...)
```

- ❶ tool_calls[0]을 사용하지 않고 for 문을 통해 tool_call을 하나씩 받아서 함수 결과를 계속 추가하도록 수정합니다.
- ❷ for tool_call in tool_calls를 추가하고 기존 코드는 4칸씩 들여쓰기를 합니다.
- ❸ GPT가 불필요하게 함수 호출을 반복하는 실수를 하기도 하므로 시스템 프롬프트를 활용해 이런 실수를 하지 않도록 가이드를 줍니다. for 문이 종료되고 나면 함수 사용을 멈추고 답변을 생성하라는 의미의 시스템 프롬프트를 messages에 추가합니다.

코드를 실행하고 여러 도시의 시간을 물어보면 get_current_time 함수를 각각 실행한 후, 그 결과를 사용해 적절한 답변을 생성해 줍니다.

사용자: 뉴욕, 런던, 파리 시간 알려줘

```
ChatCompletion(id='chatcmpl-AmfmjH7LomV789LX2kxx13sGvXmGT', choices=[Choice(finish_reason='tool_calls', index=0, logprobs=None, message=ChatCompletionMessage(content=None, refusal=None, role='assistant', audio=None, function_call=None, tool_calls=[ChatCompletionMessageToolCall(id='call_Xqxyq6I0QG1qDGdgaFtPtyvi', function=Function(arguments='{"timezone": "America/New_York"}', name='get_current_time'), type='function'), ChatCompletionMessageToolCall(id='call_OVBAqD0oAaB3KjKTFcMV9fnt', function=Function(arguments='{"timezone": "Europe/London"}', name='get_current_time'), type='function'), ChatCompletionMessageToolCall(id='call_CxY7Qg9As2FDKuVUq6l9uGls', function=Function(arguments='{"timezone": "Europe/Paris"}', name='get_current_time'), type='function') ])], created=1736163729, model='gpt-4o-2024-08-06', object='chat.completion', service_tier=None, system_fingerprint='fp_5f20662549', usage=CompletionUsage(completion_tokens=69, prompt_tokens=202, total_tokens=271, completion_tokens_details=CompletionTokensDetails(accepted_prediction_tokens=0, audio_tokens=0, reasoning_tokens=0, rejected_prediction_tokens=0), prompt_tokens_details=PromptTokensDetails(audio_tokens=0, cached_tokens=0)))
```

2025-01-06 06:42:10 America/New_York

2025-01-06 11:42:10 Europe/London

2025-01-06 12:42:10 Europe/Paris

AI: 현재 뉴욕의 시간은 2025년 1월 6일 오전 6시 42분이며, 런던의 시간은 오전 11시 42분입니다. 파리에서는 오후 12시 42분입니다.

Do it! 실습 스트림릿에서 평션 콜링 사용하기

■ 결과 파일: sec01/what_time_is_it_streamlit.py

이제 스트림릿으로 앞에서 만든 챗봇을 웹 브라우저에서 사용할 수 있도록 수정해 봅시다. what_time_is_it_terminal.py 코드에서 일부만 스트림릿 기준으로 바꿔 주면 됩니다.

1. 먼저 스트림릿을 임포트합니다.

스트림릿 임포트하기

■ what_time_is_it_streamlit.py

```
from gpt_functions import get_current_time, tools
from openai import OpenAI
from dotenv import load_dotenv
import os
import json
import streamlit as st
(... 생략 ...)
```

2. 스트림릿 스타일로 코드를 조금씩 수정합니다.

```
스트림릿 초기 시스템 메시지 설정하기 what_time_is_it_streamlit.py

(... 생략 ...)
def get_ai_response(messages, tools=None):
    (... 생략 ...)

st.title("🐼 Chatbot") ①

if "messages" not in st.session_state:
    st.session_state["messages"] = [
        {"role": "system", "content": "너는 사용자를 도와주는 상담사야."},
    ] # 초기 시스템 메시지 ②

for msg in st.session_state.messages:
    st.chat_message(msg["role"]).write(msg["content"]) ③
(... 생략 ...)
```

- ① 스트림릿 웹 페이지 상단에 제목이 나오도록 추가합니다.
- ② 스트림릿을 사용할 때 `st.session_state`에 `messages`가 없다면 대화가 시작되지 않은 상태이므로 인사말 역할을 하는 초기 메시지를 설정합니다.
- ③ 스트림릿은 파이썬 파일을 매번 새로 읽어서 실행하는 방식으로 작동하므로 기존의 대화 내용을 유지하려면 대화 내용을 계속 출력해야 합니다. 이를 위해 `st.chat_messages`를 사용하여 대화 내용을 매번 출력합니다.

3. 이어서 사용자 입력을 처리하는 코드를 추가합니다. 이 부분은 03-3절에서 만들었던 코드와 거의 동일합니다.

```
사용자 입력 처리하기 what_time_is_it_streamlit.py

(... 생략 ...)
if user_input := st.chat_input():
    st.session_state.messages.append({"role": "user", "content": user_input})
    st.chat_message("user").write(user_input) ①

    ai_response = get_ai_response(st.session_state.messages, tools=tools) ②
    print(ai_response)
    ai_message = ai_response.choices[0].message
    tool_calls = ai_message.tool_calls # AI 응답에 포함된 tool_calls 가져오기
    if tool_calls: # tool_calls가 있는 경우
        for tool_call in tool_calls:
            tool_name = tool_call.function.name # 실행해야 한다고 판단한 함수명 받기
            tool_call_id = tool_call.id # 함수 아이디 받기
```

```

arguments = json.loads(tool_call.function.arguments) # 문자열을 딕셔너리로 변환

if tool_name == "get_current_time": # tool_name이 "get_current_time"인 경우
    st.session_state.messages.append({
        "role": "function",
        "tool_call_id": tool_call_id,
        "name": tool_name,
        "content": get_current_time(timezone=arguments['timezone']),
    })

    st.session_state.messages.append({"role": "system", "content": "이제 주어진 결과를 바탕으로 답변할 차례다."})
    ai_response = get_ai_response(st.session_state.messages)
    ai_message = ai_response.choices[0].message

    st.session_state.messages.append({
        "role": "assistant",
        "content": ai_message.content
    })

print("AI\t: " + ai_message.content) # AI 응답 출력
st.chat_message("assistant").write(ai_message.content)

```

- ❶ 사용자의 입력 메시지를 `user_input`이라는 변수에 담습니다. 이 `user_input`의 값을 `st.session_state.messages`에 추가하고 `st.chat_messages`로 출력하는 과정을 거칩니다.
- ❷ GPT의 응답을 처리할 때는 `get_ai_response`를 호출하여 그 결과를 `st.session_state.messages`에 추가합니다. 기존 코드에서 `messages`로 되어 있던 내용은 모두 `st.session_state.messages`로 변경합니다.
- ❸ GPT의 답변을 `st.session_state.messages`에 추가합니다.
- ❹ 이를 `st.chat_messages`로 출력하여 스트림릿 브라우저에 대화 내용이 표시되도록 코드를 추가합니다. 앞에서는 GPT의 답변을 터미널 창에서만 출력했지만 이렇게 수정하면 스트림릿으로 띄운 브라우저에도 출력할 수 있습니다.

4. 터미널 창에서 `streamlit run 파일명.py`을 입력해 실행합니다. 다음처럼 브라우저 환경에서 GPT와 평선 콜링 기능을 활용해 채팅 할 수 있습니다.



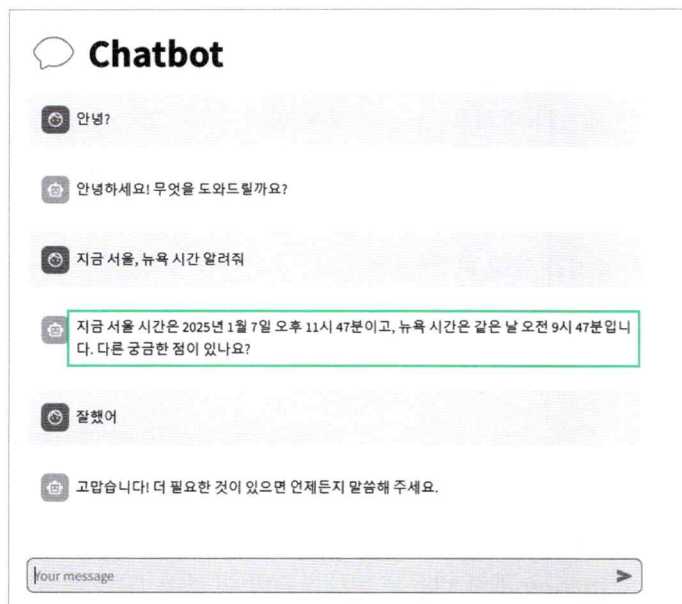
5. 현재 버전은 시스템 메시지와 함수 실행 결과가 화면에 같이 노출됩니다. 만약 시스템 메시지는 출력하고 싶지 않다면 메시지의 `role`이 `assistant` 혹은 `user`로 설정된 경우에만 출력하도록 수정하면 됩니다.

사용자와 GPT의 텍스트 답변만 출력되도록 수정하기
what_time_is_it_streamlit.py

```

(...) 생략 ...
# 대화 기록 출력
for msg in st.session_state.messages:
    if msg["role"] == "assistant" or msg["role"] == "user": # assistant 혹은 user 메시지인 경우만
        st.chat_message(msg["role"]).write(msg["content"])
(...) 생략 ...
    
```

코드를 실행하면 다음과 같이 사용자의 입력 내용과 GPT가 텍스트로 생성한 최종 답변만 출력되는 것을 확인할 수 있습니다.



07-2 GPT와 미국 주식 이야기하기

‘요새 마이크로소프트 주식 얼마야?’, ‘테슬라 주식을 지금 더 사야 할까? 아님 팔아야 할까?’ 이런 대화를 GPT와 하려면 GPT가 최신 주식 정보를 알 수 있어야 합니다. 다행히 yfinance 라는 파이썬 오픈소스 라이브러리를 사용하면 미국 주식 정보를 쉽게 가져올 수 있습니다. GPT와 대화하기에 앞서 yfinance 사용법을 익혀 보겠습니다.

Do it! 실습 yfinance 사용하기

■ 결과 파일: sec02/yfinance.ipynb

yfinance는 야후 파이낸스 [Yahoo Finance](#)의 금융 데이터를 쉽게 가져올 수 있게 해주는 오픈소스 파이썬 라이브러리입니다. 이를 사용하면 주가, 재무제표, 거래량 등 다양한 데이터를 데이터 프레임 형태로 가져올 수 있습니다. yfinance 관련 내용은 방대하지만 이 책은 GPT와 같은 대규모 언어 모델을 이용하는 데 필요한 사용법만 간단히 익히고 넘어가겠습니다.

1. 터미널 창에 다음과 같이 입력해 가상 환경에 yfinance를 설치합니다.

```
(venv) > pip install yfinance
```

2. yfinance.ipynb 파일을 생성하고 첫 번째 셀에 다음 코드를 입력하고 실행합니다. yfinance를 사용하기 쉽게 yf라는 이름으로 설치하고 마이크로소프트의 티커 [ticker](#) 객체를 생성했습니다.

◆ 여기서 'MSFT'는 마이크로소프트를 의미하는 티커입니다. 티커는 주식, ETF, 펀드 등 다양한 금융 상품을 식별하는 고유 코드입니다.

yfinance 연습하기

■ yfinance.ipynb (1)

```
import yfinance as yf

# Microsoft(MSFT)에 대한 Ticker 객체 생성
msft = yf.Ticker("MSFT")

# Ticker 객체의 정보 출력(.py에서 실행할 때는 print(msft.info)로 사용)
display(msft.info)
```

코드를 실행해 보면 msft.info가 실행되며 마이크로소프트의 기본 정보가 출력됩니다.

```
{'address1': 'One Microsoft Way',
 'city': 'Redmond',
 'state': 'WA',
 'zip': '98052-6399',
 'country': 'United States',
 'phone': '425 882 8080',
 'website': 'https://www.microsoft.com',
 'industry': 'Software - Infrastructure',
 'industryKey': 'software-infrastructure',
 'industryDisp': 'Software - Infrastructure',
 'sector': 'Technology',
 'sectorKey': 'technology',
 'sectorDisp': 'Technology',
 'longBusinessSummary': 'Microsoft Corporation develops and supports software, services,
 devices and solutions worldwide. The Productivity and Business Processes segment offers
 office, exchange, SharePoint, Microsoft Teams, office 365 Security and Compliance, Microsoft
 viva, and Microsoft 365 copilot; and office consumer services, such as Microsoft
 365 consumer subscriptions, Office licensed on-premises, and other office services.
 (... 생략 ...)'
 'grossMargins': 0.69764,
 'ebitdaMargins': 0.52804,
 'operatingMargins': 0.43143,
 'financialCurrency': 'USD',
 'trailingPegRatio': 2.2004}
```

3. 최근 5일간의 주가 정보를 확인해 보겠습니다. 여기서 **period**는 가져올 데이터의 기간을 의미합니다. 예를 들어 3일간의 데이터는 '3d', 2개월간의 데이터는 '2mo', 1년간의 데이터는 '1y'로 입력하면 됩니다.

 최근 주가 정보 보기

 yfinance.ipynb (2)

```
hist = msft.history(period="5d") # 5일간의 주가 데이터 가져오기
display(hist) # 데이터 출력
```

이 코드를 실행하면 주가 정보가 데이터프레임 형태로 나옵니다.

	Open	High	Low	Close	Volume	Dividends	Stock Splits
Date							
2025-01-15 00:00:00-05:00	419.130005	428.149994	418.269989	426.309998	19637800	0.0	0.0
2025-01-16 00:00:00-05:00	428.700012	429.489990	424.390015	424.579987	15300000	0.0	0.0
2025-01-17 00:00:00-05:00	434.089996	434.480011	428.170013	429.029999	26197500	0.0	0.0
2025-01-21 00:00:00-05:00	430.200012	430.899994	425.600006	428.500000	26085700	0.0	0.0
2025-01-22 00:00:00-05:00	437.559998	447.269989	436.000000	446.200012	27767100	0.0	0.0

이 데이터프레임에서 각 열의 정보는 다음과 같습니다.

열 이름	설명
Open(시가)	거래 시작 시점의 주가
High(고가)	해당 거래일 동안의 최고 가격
Low(저가)	해당 거래일 동안의 최저 가격
Close(종가)	해당 거래일 마지막 시점의 주가
Volume(거래량)	해당 거래일 동안 거래된 주식의 총 수량
Dividends(배당금)	주식 배당금
Stock Splits(주식 분할)	해당 거래일에 발생한 주식 분할 비율, 없으면 0으로 표기

4. 해당 종목에 대한 애널리스트들의 분석 결과도 찾아볼 수 있습니다. 다음 코드는 `msft` 객체의 `recommandations` 속성을 표시합니다. `recommandations`는 `yfinance`에서 제공하는 속성으로, 주식 분석가들이 이 주식에 대해 내린 추천(예: 매수, 매도, 보유 등)을 포함하는 판다스 데이터프레임을 반환합니다.

 주식 종목의 추천 여부 보기

yfinance.ipynb (3)

```
msft.recommandations
```

이 코드를 실행한 결과는 다음과 같습니다. period는 분석가들이 추천한 시점을 의미합니다. 0m은 현재, -1m은 1개월 전, -2m은 2개월 전을 의미합니다. 추천은 strongBuy부터 strongSell까지 5개의 등급으로 분류됩니다.

	period	strongBuy	buy	hold	sell	strongSell
0	0m	13	39	5	0	0
1	-1m	13	40	5	0	0
2	-2m	14	38	5	0	0
3	-3m	14	38	5	0	0

제가 이 코드를 실행한 시점은 2025년 2월 16일입니다. 주식 매수를 추천한다는 의견이 꾸준히 지배적이네요. 여러분이 이 코드를 실행하는 시점에는 그에 해당하는 데이터가 출력될 것입니다.

★ 한 걸음 더!

기업별 티커 알아보기

회사별로 고유한 티커 기호가 있습니다. 세계적으로 유명한 기업들의 티커는 다음과 같습니다.

회사별 티커 기호

회사	티커
아마존	"AMZN"
메타(페이스북)	"META"
구글	"GOOGL"
코카콜라	"KO"
삼성전자	"005930.KS" (한국 증권 거래소)

Do it! 실습

GPT에서 사용할 yfinance 관련 함수 만들기

■ 결과 파일: sec02/gpt_functions_0.py, sec02/stock_info_streamlit_0.py

GPT가 최신 주가 정보에 기반하여 답변하도록 하려면 필요한 기능을 함수로 만들어 평선 콜링을 활용해야 합니다. 이번 실습에서는 07-1절에서 만든 gpt_functions.py 파일을 활용해 회사의 기본 정보와 최신 주가 정보를 가져오고 투자 의견을 알려 주는 함수를 만들어 보겠습니다.

회사 기본 정보 가져오기

1. 회사 기본 정보를 가져오는 `get_yf_stock_info` 함수를 만듭니다. `yf.Ticker(ticker)` 로 정보를 가져올 회사를 선택하고, `stock.info`로 가져온 정보를 출력하고 반환합니다. `.info`는 딕셔너리 형태로 반환되어 GPT에 바로 전달할 수 없으므로 `str(info)`로 문자열로 변환합니다. 테스트로 애플의 티커인 AAPL을 활용해 main 부분에서 함수를 실행하도록 했습니다.

종목 정보를 가져오는 함수 `get_yf_info` 추가하기(1)

gpt_functions.py

```
from datetime import datetime
import pytz
import yfinance as yf

def get_current_time(timezone: str = 'Asia/Seoul'):
    (... 생략 ...)
```

```
def get_yf_stock_info(ticker: str):
    stock = yf.Ticker(ticker)
    info = stock.info
    print(info)
    return str(info)
```

```
tools = [
    (... 생략 ...)
]
```

```
if __name__ == '__main__':
    # get_current_time('America/New_York')
    info = get_yf_stock_info('AAPL')
```

이 코드를 실행하면 애플의 정보가 출력됩니다.

```
{'address1': 'One Apple Park Way', 'city': 'Cupertino', 'state': 'CA', 'zip': '95014',
'country': 'United States', 'phone': '(408) 996-1010',
(... 생략 ...)
'ebitdaMargins': 0.34437, 'operatingMargins': 0.31171, 'financialCurrency': 'USD',
'trailingPegRatio': 2.2401}
```

2. `get_yf_stock_info` 함수의 설명을 `tools`에 추가하여 GPT에서 이 함수를 사용할 수 있도록 하겠습니다. 이 함수에 대한 설명을 `description`에 쓰고 `ticker`는 필수 항목으로 지정합니다.

종목 정보를 가져오는 함수 `get_yf_info` 추가하기(2)

gpt_functions.py

```
(... 생략 ...)
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_current_time",
            (... 생략 ...)
        }
    },
    {
        "type": "function",
        "function": {
            "name": "get_yf_stock_info",
            "description": "해당 종목의 Yahoo Finance 정보를 반환합니다.",
            "parameters": {
                "type": "object",
                "properties": {
                    "ticker": {
                        "type": "string",
                        "description": "Yahoo Finance 정보를 반환할 종목의 티커를 입력하세요. (예: AAPL)",
                    }
                }
            },
            "required": ["ticker"],
        }
    }
]
(... 생략 ...)
```

이제 스트림릿에 연결된 GPT에서 `get_yf_stock_info` 함수를 사용할 수 있도록 `what_time_is_it_streamlit.py` 파일에서 코드를 수정해 보겠습니다.

3. 앞에서는 `tool_name`이 `get_current_time`인 경우만 처리되었지만 이제 `get_yf_stock_info`도 사용할 수 있도록 추가합니다. `get_current_time` 함수를 사용할 때와 동일한 방식으로 작성합니다.

get_yf_stock_info 함수 기능 추가하기

stock_info_streamlit.py

```
from gpt_functions import get_current_time, tools, get_yf_stock_info
from openai import OpenAI
from dotenv import load_dotenv
import os
import json
import streamlit as st
(... 생략 ...)

if tool_calls: # tool_calls가 있는 경우
    for tool_call in tool_calls:
        tool_name = tool_call.function.name # 실행해야 한다고 판단한 함수명 받기
        tool_call_id = tool_call.id # 함수 아이디 받기

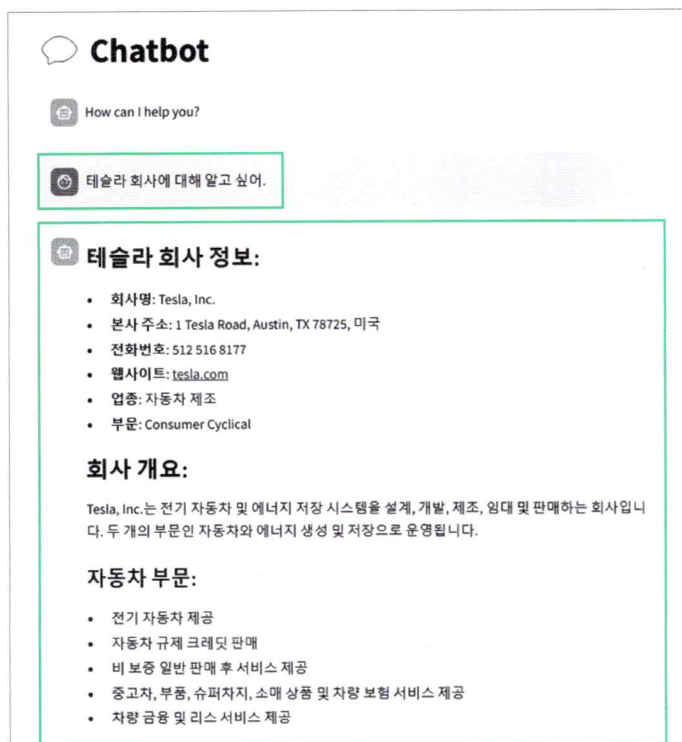
        arguments = json.loads(tool_call.function.arguments) # 문자열을 딕셔너리로 변환

        if tool_name == "get_current_time": # 만약 tool_name이 "get_current_time"이라면
            st.session_state.messages.append({
                "role": "function", # role을 "function"으로 설정
                "tool_call_id": tool_call_id,
                "name": tool_name,
                "content": get_current_time(timezone=arguments['timezone']),
            })
        elif tool_name == "get_yf_stock_info":
            st.session_state.messages.append({
                "role": "function",
                "tool_call_id": tool_call_id,
                "name": tool_name,
                "content": get_yf_stock_info(ticker=arguments['ticker']),
            })

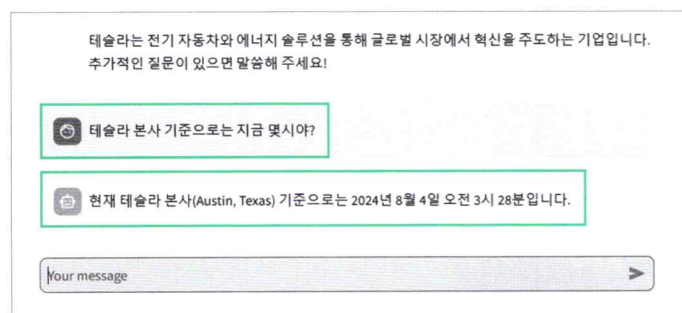
        st.session_state.messages.append({"role": "system", "content": "이제 주어진 결과를 바탕으로 답변할 차례다."})
        ai_response = get_ai_response(st.session_state.messages)
        ai_message = ai_response.choices[0].message
    (... 생략 ...)
```

4. `streamlit run 파일명.py` 명령어를 터미널 창에 입력해 코드를 실행하고, 챗봇에게 테슬라에 대해 물어보면 yfinance에서 받아온 회사 기본 정보를 기반으로 자세하게 설명해 줍니다.

◆ 언어 모델은 창의적인 답변을 하는 데 특화되어 있어서 상황에 따라 다른 답변을 생성할 수 있습니다. 정확한 서식으로 답변하는 방법은 08장에서 다룹니다.



앞에서 만든 `get_current_time` 함수도 적절하게 잘 실행하는지 확인하기 위해 테슬라 본사 기준으로 지금 몇 시인지 물어봤습니다. GPT는 이전 대화 내용에서 본사 위치가 텍사스라는 정보를 활용해 `get_current_time` 함수를 실행했습니다.



터미널 창에 출력된 `get_current_time` 함수 실행 결과를 보면 GPT는 시카고 타임존을 정확히 입력하여 결과를 반환합니다.

```
{'role': 'function', 'tool_call_id': 'call_rQn3I7u4d14zlmgOfG75B0Rr', 'name': 'get_current_time', 'content': 'America/Chicago (Austin, TX) 현재시각 2024-08-04 03:28:01 '}
```

이로써 GPT가 상황에 맞는 적절한 함수를 호출하고 필요한 값을 정확히 입력하여 실행하는 것을 확인할 수 있습니다.

Do it! 실습 코드 리팩토링하기

■ 결과 파일: `sec02/stock_info_streamlit_1.py`

사용할 함수가 점점 많아질 것이니 코드의 가독성을 위해 리팩토링을 하겠습니다. 이전 코드에서는 `tool_name`이 `get_current_time`인지 `get_yf_stock_info`인지에 따라 매번 `st.session_state.messages`에 추가하는 부분이 반복되었습니다. 하지만 호출하는 함수만 달라질 뿐 거의 동일한 코드이므로 `if` 문을 활용해 수정해 보겠습니다.

`stock_info_streamlit.py` 파일에서 `if` 문 안에서 함수를 실행한 결과를 `func_result`에 담고 `st.session_state.messages`에 추가하는 내용은 따로 분리합니다. 이렇게 하면 이전과 동일하게 작동하지만 코드는 더 깔끔해집니다. 앞으로 더 많은 함수를 추가할 때도 한두 줄만 추가하면 되겠죠.

리팩토링

■ `stock_info_streamlit.py`

```
(... 생략 ...)

if tool_name == "get_current_time": # tool_name이 "get_current_time"인 경우
    func_result = get_current_time(timezone=arguments['timezone'])

elif tool_name == "get_yf_stock_info":
    func_result = get_yf_stock_info(ticker=arguments['ticker'])

st.session_state.messages.append({
    "role": "function",
    "tool_call_id": tool_call_id,
    "name": tool_name,
    "content": func_result,
})

(... 생략 ...)
```

Do it! 실습 종목 최근 주가 정보와 추천 정보 가져오기

■ 결과 파일: sec02/gpt_functions.py, sec02/stock_info_streamlit.py

같은 방식으로 최근 주가 기록을 가져오는 함수 `get_yf_stock_history`와 추천 정보를 가져오는 함수 `get_yf_stock_recommendations`을 만들어 보겠습니다. 앞에서 만든 `gpt_functions.py`에 이어서 작업하면 됩니다.

1. `get_yf_stock_history` 함수는 2개의 매개변수 `ticker`와 `period`를 받고 `get_yf_stock_recommendations` 함수는 1개의 매개변수 `ticker`를 갖습니다. 두 함수 모두 결과를 판다스 데이터프레임 형태로 반환할 것이므로 GPT에서 사용하려면 이를 문자열로 변환해야 합니다. `str()`을 사용하여 변환할 수도 있지만 판다스 데이터프레임의 `.to_markdown()` 메서드를 활용하여 마크다운 형식의 문자열로 변환합니다. 이렇게 하면 GPT가 표 형식을 더 잘 이해할 수 있고 스트림릿에서도 데이터가 테이블 형태로 더 잘 표현됩니다. 함수가 잘 동작하는지 테스트하기 위해 `main` 부분에서 애플(APPL)에 대한 결과를 확인하는 코드를 추가합니다.

최근 주가 기록과 추천 정보를 가져오는 함수 추가하기(1)

gpt_functions.py

```
from datetime import datetime
import pytz
import yfinance as yf

def get_current_time(timezone: str = 'Asia/Seoul'):
    (... 생략 ...)
```

```
def get_yf_stock_info(ticker: str):
    (... 생략 ...)
```

```
def get_yf_stock_history(ticker: str, period: str):
    stock = yf.Ticker(ticker)
    history = stock.history(period=period)
    history_md = history.to_markdown()           # 데이터프레임을 마크다운 형식으로 변환
    print(history_md)
    return history_md
```

```
def get_yf_stock_recommendations(ticker: str):
    stock = yf.Ticker(ticker)
    recommendations = stock.recommendations
    recommendations_md = recommendations.to_markdown() # 데이터프레임을 마크다운 형식으로 변환
```

```

    print(recommendations_md)
    return recommendations_md

tools = [
    (... 생략 ...)
]

if __name__ == '__main__':
    # get_current_time('America/New_York')
    # info = get_yf_stock_history('AAPL')
)
    get_yf_stock_history('AAPL', '5d')
    print('----')
    get_yf_stock_recommendations('AAPL')

```

2. 추가한 함수를 스트림릿에서 채팅할 때 활용할 수 있도록 각 함수들을 언제 사용하고 어떤 매개변수를 사용해야 하는지 tools에 설명을 넣어야 합니다. `get_yf_stock_history`와 `get_yf_stock_recommendations` 함수에 대한 설명을 추가합니다. 이전에 추가했던 두 함수와 마찬가지로 함수의 이름과 설명을 덧붙이고 각 함수에 필요한 매개변수를 정의합니다. 특히 `get_yf_stock_history` 함수는 ticker 외에도 기간을 나타내는 `period`를 문자열 형식으로 받아야 합니다. `period`를 `yfinance`에서 지원하는 양식에 맞춰야 하므로 GPT가 이를 자동으로 생성할 수 있도록 예시를 `description`에 포함시킵니다.

최근 주가 기록과 추천 정보를 가져오는 함수 추가하기(2)

gpt_functions.py

```

(... 생략 ...)
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_current_time",
            (... 생략 ...)
        }
    },
    {
        "type": "function",
        "function": {
            "name": "get_yf_stock_info",
            (... 생략 ...)
        }
    },
]

```

```

{
  "type": "function",
  "function": {
    "name": "get_yf_stock_history",
    "description": "해당 종목의 Yahoo Finance 주가 정보를 반환합니다.",
    "parameters": {
      "type": "object",
      "properties": {
        "ticker": {
          "type": "string",
          "description": "Yahoo Finance 주가 정보를 반환할 종목의 티커를 입력
하세요. (예: AAPL)',
        },
        "period": {
          "type": "string",
          "description": "주가 정보를 조회할 기간을 입력하세요. (예: 1d, 5d,
1mo, 1y, 5y)',
        },
      },
      "required": ['ticker', 'period'],
    },
  },
},
{
  "type": "function",
  "function": {
    "name": "get_yf_stock_recommendations",
    "description": "해당 종목의 Yahoo Finance 추천 정보를 반환합니다.",
    "parameters": {
      "type": "object",
      "properties": {
        "ticker": {
          "type": "string",
          "description": "Yahoo Finance 추천 정보를 반환할 종목의 티커를 입력
하세요. (예: AAPL)',
        },
      },
      "required": ['ticker'],
    },
  },
},
]
(... 생략 ...)

```

코드를 처음 실행하면 다음처럼 오류 메시지가 나올 수 있습니다. `.to_markdown()` 메서드를 사용해 데이터프레임을 마크다운 형식으로 변환하는 과정에 필요한 `tabulate` 라이브러리가 현재 파이썬 가상 환경에 설치되어 있지 않아서 발생하는 오류입니다.

```
(... 생략 ...)
ModuleNotFoundError: No module named 'tabulate'
(... 생략 ...)
ImportError: Missing optional dependency 'tabulate'. Use pip or conda to install tabulate.
(... 생략 ...)
```

3. 가상 환경에서 `tabulate`를 설치합니다.

```
(venv) > pip install tabulate
```

다시 `gpt_functions.py`를 실행해 보면 다음처럼 터미널 창에 결과가 출력됩니다. 구체적인 수치나 내용은 최신 자료에 맞게 업데이트되어 있을 것입니다.

Date	Open	High	Low	Close	Volume	Dividends	Stock Splits
2024-07-29 00:00:00-04:00	216.96	219.3	215.75	218.24	3.63118e+07	0	0
2024-07-30 00:00:00-04:00	219.19	220.33	216.12	218.8	4.16438e+07	0	0
2024-07-31 00:00:00-04:00	221.44	223.82	220.63	222.08	5.00363e+07	0	0
2024-08-01 00:00:00-04:00	224.37	224.48	217.02	218.36	6.2501e+07	0	0
2024-08-02 00:00:00-04:00	219.15	225.6	217.71	219.86	9.84801e+07	0	0

period	strongBuy	buy	hold	sell	strongSell
0 0m	11	21	6	0	0
1 -1m	12	19	12	1	0
2 -2m	10	19	13	1	0
3 -3m	10	24	7	1	0

4. 이제 GPT API를 사용하는 스트림릿 코드에서도 함수들을 사용할 수 있도록 수정하겠습니다. 필요한 함수들을 импорт합니다. GPT가 `gpt_functions`에 새로 정의한 함수를 사용하여 한다고 판단하는 경우를 처리하기 위해 `elif` 조건문을 추가하고 GPT가 해당 함수를 실행할 때 필요한 매개변수를 함께 제공하도록 수정합니다.

```

from gpt_functions import get_current_time, tools, get_yf_stock_info, get_yf_stock_
history, get_yf_stock_recommendations # 필요한 함수 임포트
from openai import OpenAI
from dotenv import load_dotenv
import os
import json
import streamlit as st

(... 생략 ...)

    if tool_name == "get_current_time": # tool_name이 "get_current_time"인 경우
        func_result = get_current_time(timezone=arguments['timezone'])

    elif tool_name == "get_yf_stock_info":
        func_result = get_yf_stock_info(ticker=arguments['ticker'])

    elif tool_name == "get_yf_stock_history": # get_yf_stock_history 함수 호출
        func_result = get_yf_stock_history(
            ticker=arguments['ticker'],
            period=arguments['period']
        )

    elif tool_name == "get_yf_stock_recommendations": # get_yf_stock_recom-
mendations 함수 호출
        func_result = get_yf_stock_recommendations(
            ticker=arguments['ticker']
        )

    st.session_state.messages.append({
        "role": "function",
        "tool_call_id": tool_call_id,
        "name": tool_name,
        "content": func_result,
    })

(... 생략 ...)

```

streamlit run 파일명.py 명령어로 코드를 실행합니다. 챗봇에 테슬라 주식이 한 달 전과 비교해 어떤 상태인지 물어보니 get_yf_stock_history 함수를 실행시키고 ‘한달 전에 비해 주가가 하락했을 뿐만 아니라 변동성도 커졌다’고 답변합니다. 또한 이 함수의 결과도 마크다운 형식으로 반환되어 스트림릿에서 표 형태로 깔끔하게 표시됩니다.

◆ 만약 위에서 if msg["role"] == "assistant" or msg["role"] == "user": 와 같이 메시지의 role이 assistant나 user인 경우에만 출력되도록 해놓은 상태라면 함수 실행 결과인 표는 출력되지 않습니다

```
ChatCompletion(id='chatcpl-At7pcojZHw2c8u5p16AsHXYFGJo1S',
(... 생략 ...)
function=Function(arguments='{"ticker":"TSLA","period":"1mo"}', name='get_yf_stock_history'), type='function'))], ... 생략 ...
```

테슬라 주식이 한달전에 비해 어때?

F

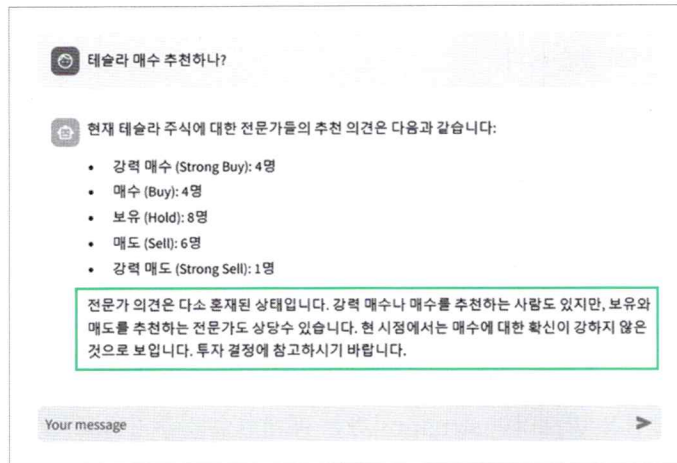
Date	Open	High	Low	Close	Volume	Dividends	Stock Splits
2024-07-03 00:00:00-04:00	234.56	248.35	234.25	246.39	1.66562e+08	0	0
2024-08-02 00:00:00-04:00	214.88	216.13	205.78	207.67	8.25522e+07	0	0

한 달 전, 즉 2024년 7월 3일 테슬라의 주가가 (234.56) 달러에 개장하여 (246.39) 달러에 마감되었습니다. 현재, 즉 2024년 8월 2일 테슬라의 주가는 (216.86) 달러에 개장하여 (207.67) 달러에 마감되었습니다.

테슬라 주가는 한 달 전과 비교하여 약 (246.39 - 207.67 = 38.72) 달러 하락하였습니다. 금액뿐만 아니라 주가의 변동폭도 커진 것을 확인할 수 있습니다.

그리고 해당 주식의 매수를 추천하는지 물어보니 `get_yf_stock_recommendations` 함수를 실행시키고 '매수에 대한 전문가들의 확신이 적은 상태'라고 답변합니다.

```
ChatCompletion(id='chatcmpl-At7qDZqZkkCg0m805giJHT0dfki7S',
(... 생략 ...)
function=Function(arguments='{"ticker":"TSLA"}', name='get_yf_stock_recommendations'),
type='function'))], ... 생략 ...)
```



07-3 스트림 출력하기

현재 응답은 GPT API에서 한 번에 완성해서 반환합니다. 사용자가 원하는 대로 작동하지만 답변이 나올 때까지 시간이 오래 걸립니다. 챗GPT나 Bing처럼 생성된 답변이 타이핑하듯이 출력되면 좋겠습니다. 이런 방식을 스트림 출력이라고 합니다. 이번 절에서는 GPT API에서 스트림 출력을 구현하는 방법을 알아보겠습니다.

GPT API 자체 기능과 스트림릿의 기능을 분리해서 이해할 수 있도록 스트림 출력 방식을 터미널 창과 스트림릿으로 나눠서 설명하겠습니다. 원리를 알아 두면 스트림릿 없이 프로그램이나 서비스를 개발할 때도 응용할 수 있을 겁니다.

Do it! 실습 터미널 창에서 스트림 방식으로 출력하기

■ 결과 파일: sec03/stock_info_streaming_0.py

GPT API에서 스트림 방식으로 답변을 받으려면 `stream`이라는 매개변수를 추가하고 `True`로 설정해야 합니다. 평선 콜링을 하지 않는 단순한 텍스트 답변은 비교적 쉽게 처리할 수 있으므로 일반 텍스트 답변에 먼저 적용해 봅시다. GPT에서 스트림 출력을 확인할 수 있도록 `stock_info_streamlit.py`을 수정해 보겠습니다.

1. `get_ai_response` 함수에 매개변수 `stream`을 추가하고 기본값을 `True`로 설정합니다. 여기에 사용된 `client.chat.completions.create`에 `stream=stream` 인자를 추가합니다. `client.chat.completions.create`의 기본 설정이 `stream=False`로 되어 있어서 이전에는 GPT가 답변을 완성한 후 결과를 한 번에 반환했습니다. 이렇게 수정하면 GPT가 스트림 출력 방식을 이용해 응답을 타이핑하듯 실시간으로 전송하게 됩니다. 일반적으로 사용하는 `return`은 한 번에 결과를 반환하는 방식이므로 실시간 출력에는 적합하지 않아 `yield`로 수정합니다. `stream`이 `False`인 경우에는 기존 방식대로 결과를 한 번에 반환하지만, `True`인 경우에는 `for` 문을 사용해 중간중간 값을 `yield`로 반환합니다.

(... 생략 ...)

```
def get_ai_response(messages, tools=None, stream=True):
    response = client.chat.completions.create(
        model="gpt-4o",      # 응답 생성에 사용할 모델 지정
        stream=stream,       # 스트리밍 출력을 위해 설정
        messages=messages,   # 대화 기록을 입력으로 전달
        tools=tools,         # 사용할 수 있는 도구 목록 전달
    )

    if stream:
        for chunk in response:
            yield chunk # 생성된 응답의 내용을 yield로 순차적으로 반환
    else:
        return response # 생성한 응답 내용 반환

(... 생략 ...)
```

yield 키워드는 함수를 한 번에 끝까지 실행하지 않고 중간중간 값을 내보낼 수 있게 만들어 줍니다. 이 키워드를 사용한 코드를 실행하면 함수가 결과를 하나씩 반환하고 멈춘 뒤 필요할 때 다시 이어서 실행됩니다. 이런 함수를 제너레이터라고 하는데, 오래 걸리는 작업을 순차로 처리할 때 유용합니다. 이 실습에서는 GPT가 답변을 완성하는 데 시간이 오래 걸리므로 응답을 쪼개어 나눠서 반환하려고 yield를 활용했습니다.

2. 이전에는 ai_response를 가져와서 곧바로 사용할 수 있었지만 이제 응답이 청크 단위로 쪼개져 순차적으로 넘어오므로 바로 사용할 수 없습니다. 우선 이 코드를 주석 처리하고 for 문을 이용해 ai_response를 출력해 봅시다. 코드를 아직 완성하지 않았지만 터미널 창에 어떻게 출력되는지 먼저 살펴보겠습니다.

```
if user_input := st.chat_input():
    st.session_state.messages.append({"role": "user", "content": user_input}) # 사용자
    메시지를 대화 기록에 추가
    st.chat_message("user").write(user_input)
```

```

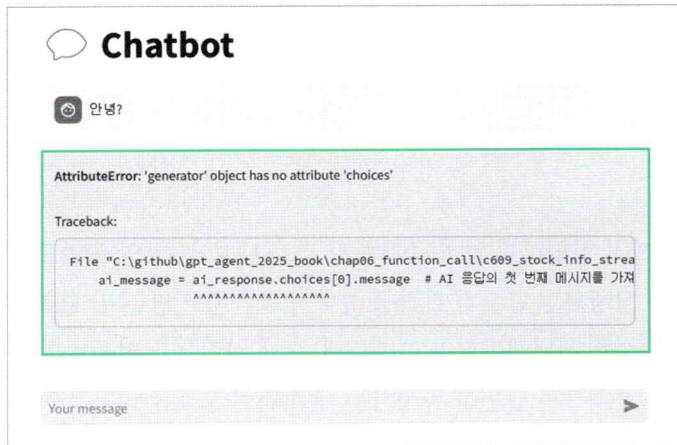
ai_response = get_ai_response(st.session_state.messages, tools=tools) # 대화 기록
을 기반으로 AI 응답 가져오기
# print(ai_response) # gpt에서 반환되는 값을 파악하기 위해 임시로 추가

for chunk in ai_response:
    print(chunk)
print('=====')

ai_message = ai_response.choices[0].message # AI 응답의 첫 번째 메시지 가져오기

```

3. 현재 상태에서 터미널 창에서 `streamlit run 파일명.py`으로 코드를 실행하고 웹 브라우저에서 간단한 문장을 입력하면 다음처럼 오류 화면이 나옵니다.



이 오류는 차차 살펴보도록 하고 터미널 창에 출력된 결과를 봅시다. ChatCompletionChunk 라는 객체로 결과가 출력되었습니다. 그 객체 안에는 choices라는 항목이 있고 그 안의 Choice 객체의 delta 값에 ChoiceDelta라는 객체가 들어 있습니다. 이 ChoiceDelta 객체의 content 항목에는 GPT가 생성한 메시지가 포함되어 있는데, 이 메시지가 청크 단위로 분리되어 순차로 넘어왔습니다. 이 조각난 청크가 스트림 방식으로 터미널 창에 출력되도록 코드를 수정해 보겠습니다.

```

ChatCompletionChunk(id='chatcmpl-AnM5sXAHvP259uhGijxM2kTgPHTVKf', choices=[Choice(delta=ChoiceDelta(content='', function_call=None, refusal=None, role='assistant', tool_calls=None), finish_reason=None, index=0, logprobs=None)], created=1736329438, model='gpt-4o-2024-08-06', object='chat.completion.chunk', service_tier=None, system_fingerprint='fp_d28bcae782', usage=None)
ChatCompletionChunk(id='chatcmpl-AnM5sXAHvP259uhGijxM2kTgPHTVKf', choices=[Choice(delta=ChoiceDelta(content='안', function_call=None, refusal=None, role=None, tool_calls=None), finish_reason=None, index=0, logprobs=None)], created=1736329438, model='gpt-4o-2024-08-06', object='chat.completion.chunk', service_tier=None, system_fingerprint='fp_d28bcae782', usage=None)
ChatCompletionChunk(id='chatcmpl-AnM5sXAHvP259uhGijxM2kTgPHTVKf', choices=[Choice(delta=ChoiceDelta(content='녕하세요', function_call=None, refusal=None, role=None, tool_calls=None), finish_reason=None, index=0, logprobs=None)], created=1736329438, model='gpt-4o-2024-08-06', object='chat.completion.chunk', service_tier=None, system_fingerprint='fp_d28bcae782', usage=None)
ChatCompletionChunk(id='chatcmpl-AnM5sXAHvP259uhGijxM2kTgPHTVKf', choices=[Choice(delta=ChoiceDelta(content='!', function_call=None, refusal=None, role=None, tool_calls=None), finish_reason=None, index=0, logprobs=None)], created=1736329438, model='gpt-4o-2024-08-06', object='chat.completion.chunk', service_tier=None, system_fingerprint='fp_d28bcae782', usage=None)
ChatCompletionChunk(id='chatcmpl-AnM5sXAHvP259uhGijxM2kTgPHTVKf', choices=[Choice(delta=ChoiceDelta(content='무엇', function_call=None, refusal=None, role=None, tool_calls=None), finish_reason=None, index=0, logprobs=None)], created=1736329438, model='gpt-4o-2024-08-06', object='chat.completion.chunk', service_tier=None, system_fingerprint='fp_d28bcae782', usage=None)

```

4. 터미널 창에서 결과가 스트림 방식으로 출력되도록 코드를 다음과 같이 수정합니다.

```
터미널 창에 스트림 방식으로 출력하기 stock_info_streamlit.py

(... 생략 ...)
if user_input := st.chat_input():
    st.session_state.messages.append({"role": "user", "content": user_input}) # 사용자
    메시지를 대화 기록에 추가
    st.chat_message("user").write(user_input)

    ai_response = get_ai_response(st.session_state.messages, tools=tools) # 대화 기록을
    기반으로 AI 응답 가져오기
    # print(ai_response) # gpt에서 반환되는 값을 파악하기 위해 임시로 추가
    content = '' ❶
    for chunk in ai_response:
        content_chunk = chunk.choices[0].delta.content ❷
        if content_chunk:
            print(content_chunk, end="") ❸
            content += content_chunk ❹
        else:
            continue

    print('\n=====')
    print(content)

ai_message = ai_response.choices[0].message
(... 생략 ...)
```

- ❶ 비어 있는 문자열을 content라는 변수명으로 선언합니다. 이 변수는 for 문을 반복하면서 각 청크의 content 내용을 덧붙여 나가는 용도로 사용합니다.
- ❷ 이어서 청크 속에서 content를 추출하는 코드를 작성합니다. 앞서 살펴보았듯이 get_ai_response 함수에서 stream을 True로 설정하면 yield로 청크 단위가 쪼개져서 반환됩니다. for 문에서 이 결과를 chunk라는 변수에 담아 반복하고 있으므로 choice[0]의 delta에 담긴 content를 가져와 content_chunk 변수에 담습니다.
- ❸ if 문에서는 content_chunk가 None이 아니라면 그 값을 content에 사용하는 코드가 실행됩니다. 만약 평선 콜링을 해야 하거나 content_chunk가 비어 있다면 이를 건너뛰고 처리하지 않습니다.
- ❹ content_chunk가 있다면 그 내용을 터미널 창에 출력합니다. 일반적인 print는 출력한 후 줄 바꿈을 하므로 end=""로 줄바꿈 없이 출력 결과가 계속 이어지도록 합니다. 이렇게 하면 마치 타이핑하듯이 터미널 창에 출력됩니다.
- ❺ 그리고 content_chunk를 content에 덧붙여 나갑니다.

현재 상태에서 `streamlit run 파일명.py`로 실행해 봅시다. 여전히 오류가 발생하지만 터미널 창에 스트림 방식으로 출력되는 결과를 확인할 수 있습니다.

K-pop 산업은 전 세계적으로 큰 인기를 끌고 있지만, 몇 가지 문제점과 도전 과제도 안고 있습니다. 여기 몇 가지 주요 문제점을 살펴보겠습니다.

1. ****과도한 스케줄과 스트레스****: K-pop 아이돌들은 종종 과도한 스케줄로 인해 극심한 피로와 스트레스를 겪습니다. 데뷔 전부터 시작되는 긴 연습 시간과 데뷔 후의 빡빡한 일정이 이들의 정신적, 육체적 건강에 부정적인 영향을 미칠 수 있습니다.
2. ****엄격한 이미지 관리****: 많은 K-pop 아이돌들은 엄격한 이미지 관리를 요구받습니다. 특정 체중과 외모 기준을 유지해야 하며, 이는 종종 식이장애와 같은 건강 문제를 야기할 수 있습니다.

2. ****엄격한 이미지 관리****: 많은 K-pop 아이돌들은 엄격한 이미지 관리를 요구받습니다. 특정 체중과 외모 기준을 유지해야 하며, 이는 종종 식이장애와 같은 건강 문제를 야기할 수 있습니다.

3. ****사생활 침해****: 팬과 대중의 과도한 관심으로 사생활 침해 문제가 발생할 수 있습니다. 연예인으로서 많은 관심을 받지만, 사적인 삶을 유지하기 어려운 경우가 많습니다.

4. ****계약 문제 및 불공정 대우****: 일부 아이돌 그룹은 회사와의 불공정한 계약, 수익 분배 문제 등으로 어려움을 겪습니다. 긴 계약 기간, 낮은 수익 배분 등은 종종 논란이 됩니다.

5. ****인종 및 문화적 차별****: K-pop이 세계적으로 확산됨에 따라, 문화적 오해나 인종적 감수성 결여와 관련된 문제도 발생할 수 있습니다.

6. ****팬 문화의 극단성****: 열정적인 팬덤이 K-pop의 성공에 중요한 역할을 하지만, 일부 극성 팬의 행동은 문제가 될 수 있습니다. 이러한 행동은 대상 아이돌에게 스트레스를 줄 뿐만 아니라, 다른 팬들과의 갈등을 일으킬 수도 있습니다.

7. ****자기표현의 제한****: 많은 K-pop 아이돌들은 자신을 자유롭게 표현하는 데 제한이 있으며, 이는 창의성과 개인의 잠재력을 억누를 수 있습니다.

K-pop 산업은 이러한 문제들을 해결하기 위해 지속적인 개선 노력이 필요하며, 아이돌의 인권과 복지를 보장하는 것이 중요합니다.

K-pop 산업은 전 세계적으로 큰 인기를 끌고 있지만, 몇 가지 문제점과 도전 과제도 안고 있습니다. 여기 몇 가지 주요 문제점을 살펴보겠습니다.

1. ****과도한 스케줄과 스트레스****: K-pop 아이돌들은 종종 과도한 스케줄로 인해 극심한 피로와 스트레스를 겪습니다. 데뷔 전부터 시작되는 긴 연습 시간과 데뷔 후의 빡빡한 일정이 이들의 정신적, 육체적 건강에 부정적인 영향을 미칠 수 있습니다.

답변이 짧으면 스트림 방식으로 출력되는지 확인하기 어려우므로 복잡하고 어려운 질문을 해보세요. 저는 'KPOP의 문제점을 이야기해 봐'라고 질문했습니다. 터미널 창을 보면 꽤 긴 지하고 긴 메시지가 스트림 방식으로 타이핑하듯이 출력됩니다. 그리고 '=====' 아래에 응답을 다 합친 content가 다시 한번 출력됩니다. 아직 오류가 발생하지만 답변이 청크 단위로 출력되는 스트림 방식을 조금씩 이해할 수 있을 것입니다.

Do it! 실습 스트림릿에서 스트림 방식으로 출력하기

■ 결과 파일: `sec03/stock_info_streaming_1.py`

터미널 창뿐만 아니라 스트림릿에서도 스트림 방식으로 출력되도록 수정해 보겠습니다.

1. `with`를 사용해 비어 있는 채트 메시지를 만들고 그 메시지를 `assistant`로 설정합니다. 그 안에서 기존 코드를 반복하며 그 시점까지 합친 `content`를 비어 있는 채트 메시지에 마크다운 형태로 채워 주는 방식입니다. 무슨 말인지 이해하기 어려울 수 있지만, 이 부분은 파이썬이나 GPT API의 동작과 직접 관련되지 않고 스트림릿의 특정 규칙에 따라 설정한다고 이해하면 됩니다.

```
(... 생략 ...)
content = ''

with st.chat_message("assistant").empty(): # 스트림릿 챗 메시지 초기화
    for chunk in ai_response:
        content_chunk = chunk.choices[0].delta.content
        if content_chunk:
            print(content_chunk, end='')
            content += content_chunk
            st.markdown(content) # 스트림릿 챗 메시지에 마크다운으로 출력

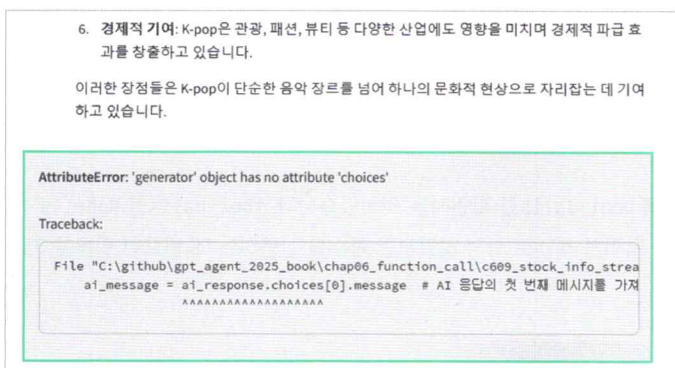
print('\n=====')
print(content)
(... 생략 ...)
```

4칸 들여쓰기

이 코드를 실행하면 스트림릿에서 답변이 타이핑하듯 스트림 방식으로 출력됩니다.



답변이 다 출력된 이후에는 여전히 오류가 발생하고 있습니다. 이제 이 문제를 해결해 보겠습니다.



오류를 해결하는 방식은 간단합니다. 지금까지 오류가 발생한 이유를 터미널 창과 스트림릿 화면에서 친절히 안내해 주고 있었습니다. 이제 GPT가 결과를 스트림 방식으로 반환하므로 `ai_message = ai_response.choices[0].message` 코드가 더 이상 맞지 않게 된 것이죠.

2. 코드가 맞지 않게 된 부분을 삭제하거나 주석 처리하고 영향을 받는 몇 가지 부분을 수정하겠습니다.

오류 없애기

stock_info_streamlit.py

```
(... 생략 ...)
if user_input := st.chat_input():
    st.session_state.messages.append({"role": "user", "content": user_input})
    st.chat_message("user").write(user_input)

    ai_response = get_ai_response(st.session_state.messages, tools=tools)
    # print(ai_response) # gpt에서 반환되는 값을 파악하기 위해 임시로 추가
    content = ''
    tool_calls = None ❶

    with st.chat_message("assistant").empty(): # 스트림릿 챗 메시지 초기화
        (... 생략 ...)
        print('\n=====')
        print(content)

        # ai_message = ai_response.choices[0].message
        # tool_calls = ai_message.tool_calls
        if tool_calls: # tool_calls가 있는 경우
            (... 생략 ...)

        st.session_state.messages.append({
            "role": "assistant",
            "content": content ❸
        }) # 대화 기록에 AI 응답 추가

        print("AI\t: " + content) ❹
        # st.chat_message("assistant").write(content) ❺
```

❶ 해당 코드의 목적은 평선 콜링을 할 때 `tool_calls`를 확인하는 것이었습니다. `tool_calls`를 `None`으로 초기화합니다. 아직은 평선 콜링을 위한 처리를 제대로 하지 않았지만 함수를 실행하는 데 필요한 질문을 하지 않는다면 문제없이 실행됩니다.

- 2 GPT를 stream=True로 설정하면 항상 오류가 발생하던 부분입니다. 평선 콜링을 판단하는 tool_calls를 얻기 위한 코드였으므로 삭제하거나 주석 처리합니다.
- 3 원래는 ai_message.content로 되어 있었지만 이제 스트림 방식으로 content에 content_chunk를 덧붙이는 방식으로 변경되었습니다. 스트림 방식으로 스트림릿에 직접 출력하므로 출력하는 코드도 수정합니다.
- 4 ai_message.content로 되어 있던 부분은 content로 수정하고 스트림릿에 출력하는 코드는 주석 처리하거나 삭제합니다.

이 코드를 실행해 보면 오류 없이 대화를 이어 나갈 수 있습니다.

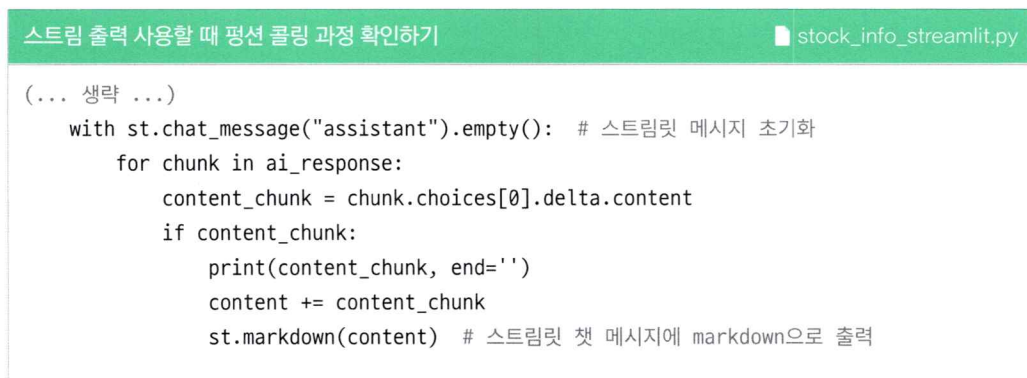


Do it! 실습 스트림 방식에서 평선 콜링 사용하기

■ 결과 파일: sec03/stock_info_streamlit.py

stream이 True인 상태에서 평선 콜링을 사용하면 그 결과도 조각 단위로 반환됩니다. 이 부분은 설명하기보다 직접 터미널 창에서 결과를 출력해 보면 더 쉽게 이해할 수 있습니다. 앞의 실습에 이어서 stock_info_streaming.py 파일을 계속 수정해서 사용하겠습니다.

1. 다음처럼 chunk를 출력하는 코드를 임시로 추가합니다.



```
print(chunk) # 임시로 chunk 출력

print('\n=====')
print(content)
(... 생략 ...)
```

터미널 창에 `streamlit run 파일명.py`을 입력해 실행하고 `get_current_time` 함수를 사용하도록 유도하는 질문을 하니 결과가 출력되지 않습니다.



터미널 창에 출력된 결과를 살펴봅시다. `Content`는 `None`으로 표시되지만 `tool_calls`에는 정보가 포함되어 있습니다. 이 정보에는 어떤 함수를 사용할지를 `name='get_current_time'`로 나타냈습니다. 또한 이 함수를 사용하는 데 필요한 매개변수들이 `arguments` 안에 스트림 방식으로 조금씩 출력되고 있습니다.

```
(... 생략 ...)
=====
안녕하세요! 무엇을 도와드릴까요?
AI: 안녕하세요! 무엇을 도와드릴까요?
ChatCompletionChunk(id='chatcmpl-AnPkiG1ybEo03gXzN4yE9mNlJLgx4', choices=[Choice(delta=ChoiceDelta(content=None, function_call=None, refusal=None, role='assistant', tool_calls=[ChoiceDeltaToolCall(index=0, id='call_st9HVMYIRYjznSSTb75z3xC6', function=ChoiceDeltaToolCallFunction(arguments='', name='get_current_time', type='function'))], finish_reason=None, index=0, logprobs=None)], created=1736340428, model='gpt-4o-2024-08-06', object='chat.completion.chunk', service_tier=None, system_fingerprint='fp_d28bcae782', usage=None)
ChatCompletionChunk(id='chatcmpl-AnPkiG1ybEo03gXzN4yE9mNlJLgx4', choices=[Choice(delta=ChoiceDelta(content=None, function_call=None, refusal=None, role=None, tool_calls=
```

```
[ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments=
'{"', name=None), type=None))], finish_reason=None, index=0, logprobs=None)], ... 생략
...
ChatCompletionChunk(id='chatcmpl-AnPkiG1ybEo03gXzN4yE9mNlJLgx4', choices=[Choice(delta=
ChoiceDelta(content=None, function_call=None, refusal=None, role=None, tool_calls=
[ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments=
'timezone', name=None), type=None))], finish_reason=None, index=0, logprobs=None)], ...
생략 ...
```

2. 이렇게 출력되는 정보를 한데 모으기 위해 코드를 수정하겠습니다.

평션 콜링 관련 청크 수집하기

stock_info_streamlit.py

```
(... 생략 ...)
ai_response = get_ai_response(st.session_state.messages, tools=tools) # 대화 기록
을 기반으로 AI 응답 가져오기
# print(ai_response) # gpt에서 반환되는 값을 파악하기 위해 임시로 추가
content = ''
tool_calls = None # tool_calls 초기화
tool_calls_chunk = [] ①

with st.chat_message("assistant").empty(): # 스트림릿 챗 메시지 초기화
    for chunk in ai_response:
        content_chunk = chunk.choices[0].delta.content
        if content_chunk:
            print(content_chunk, end='')
            content += content_chunk
            st.markdown(content) # 스트림릿 챗 메시지에 markdown으로 출력

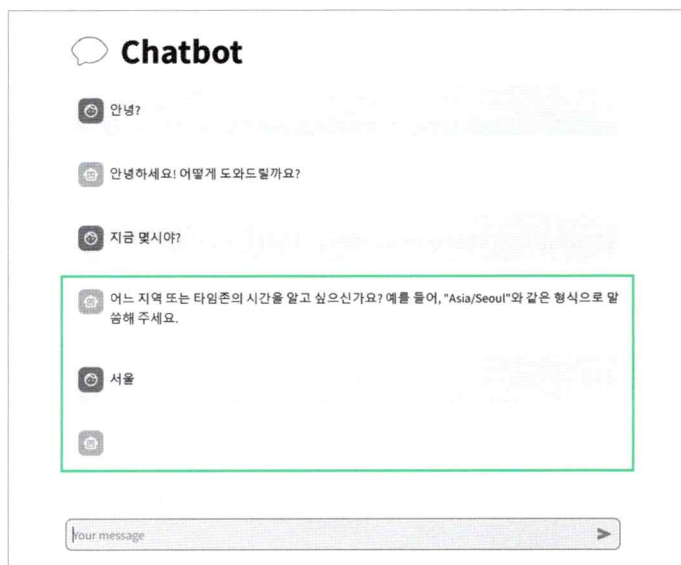
        # print(chunk) ②
        if chunk.choices[0].delta.tool_calls:
            tool_calls_chunk += chunk.choices[0].delta.tool_calls ③

    print('\n=====')
    print(content)

    print('\n===== tool_calls_chunk')
    for tool_call_chunk in tool_calls_chunk: ④
        print(tool_call_chunk)
(... 생략 ...)
```

- 1 tool_calls_chunk를 비어 있는 리스트로 초기화합니다. 파편화된 청크 데이터를 담아 두는 변수입니다.
- 2 앞에서 임시로 추가한 print(chunk)는 이제 필요하지 않으므로 주석 처리하거나 삭제합니다.
- 3 tool_calls가 있는 경우에는 초기화한 tool_calls_chunk에 덧붙여 나갑니다.
- 4 이렇게 만든 tool_calls_chunk 정보를 출력합니다.

파일을 실행하고 다음처럼 get_current_time 함수를 실행하는 질문을 입력하면 해당 함수가 필요한 질문에는 여전히 출력되지 않습니다.



하지만 터미널 창에 출력된 결과는 다음과 같습니다. tool_call_chunks 리스트에 어떤 함수를 실행해야 하는지와 그 인자는 어떻게 입력해야 하는지에 관한 정보가 쪼개진 청크 상태로 담겨 있습니다. 이 정보를 잘 합쳐서 get_current_time과 {"timezone": "Asia/Seoul"}을 만들어 내면 됩니다.

```

===== tool_calls_chunk
ChoiceDeltaToolCall(index=0, id='call_tj7svFo160AWYjM14tf4QBUX', function=ChoiceDeltaToolCallFunction(arguments='', name='get_current_time'), type='function')
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='{', name=None), type=None)
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='timezone', name=None), type=None)
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments=':', name=None), type=None)
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='Asia', name=None), type=None)
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='/', name=None), type=None)
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='Se', name=None), type=None)
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='oul', name=None), type=None)
ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='}'), name=None), type=None)
AI :

```

3. 이제 조각난 정보를 모아 딕셔너리 형태로 정리하기 위해 `tool_list_to_tool_obj` 함수를 만들어 보겠습니다. 이 함수는 `tools`로 전달된 조각 단위의 정보를 리스트 형태로 받아 딕셔너리 형태의 리스트로 반환합니다. 리스트 형태로 반환하는 이유는 GPT가 여러 함수를 동시에 실행하려고 할 때 해당 내용을 리스트에 담기 위함입니다. 예를 들어 ‘테슬라 주식 정보 알려줘’라고 질문하면 GPT는 3가지 함수 `get_yf_stock_info`, `get_yf_stock_history`, `get_yf_recommendation`를 연속으로 실행하려고 할 수 있습니다.

◆ 이 함수는 오픈AI 커뮤니티를 참고했습니다(출처: <https://community.openai.com/t/help-for-function-calls-with-streaming/627170/2>).

조각난 정보를 모아 딕셔너리 형태로 정리하기(1)

■ stock_info_streamlit.py

```
from gpt_functions import get_current_time, tools, get_yf_stock_info, get_yf_stock_history, get_yf_stock_recommendations
from openai import OpenAI
from dotenv import load_dotenv
import os
import json
import streamlit as st
from collections import defaultdict  ❶

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기

client = OpenAI(api_key=api_key) # 오픈AI 클라이언트의 인스턴스 생성

def tool_list_to_tool_obj(tools):
    tool_calls_dict = defaultdict(lambda: {"id": None, "function": {"arguments": "", "name": None}, "type": None})

    for tool_call in tools:
        # id가 None이 아닌 경우 설정
        if tool_call.id is not None:
            tool_calls_dict[tool_call.index]["id"] = tool_call.id

        # 함수 이름이 None이 아닌 경우 설정
        if tool_call.function.name is not None:
            tool_calls_dict[tool_call.index]["function"]["name"] = tool_call.function.name

        # 인자 추가
        tool_calls_dict[tool_call.index]["function"]["arguments"] += tool_call.function.arguments

        # 타입이 None이 아닌 경우 설정
```

❷

```

        if tool_call.type is not None:
            tool_calls_dict[tool_call.index]["type"] = tool_call.type

    tool_calls_list = list(tool_calls_dict.values())

    return {"tool_calls": tool_calls_list} # 반환
(... 생략 ...)

```

- ❶ 파이썬 내장 모듈인 collections에서 defaultdict 클래스를 가져옵니다. 이 클래스를 사용하면 기본값을 설정해 놓은 딕셔너리를 생성할 수 있어 존재하지 않는 키에 접근할 때 자동으로 기본값이 할당됩니다.
- ❷ 도구(함수) 호출에 관한 조각 정보들은 tools라는 매개변수에 리스트 형태로 전달해 오므로 이를 반복문으로 처리합니다. 반복문 내에서 id, function.name, function.arguments, type이 None이 아닌 경우에만 tool_calls_dict에 해당하는 값을 채웁니다.
- ❸ tool_calls_dict에 채워진 값들을 리스트로 변환하여 tool_calls_list에 저장하고 최종적으로 return {"tool_calls": tool_calls_list} 형태로 반환합니다.

4. 이제 이 함수를 사용해 보겠습니다. 기존에 tool_call_chunks를 출력해서 확인했던 코드는 삭제합니다. 새로 만든 tool_list_to_tool_obj 함수를 이용해 tool_calls_chunk를 결합하여 스트림을 하지 않은 상태와 최대한 유사하게 만들어 tool_obj에 담습니다. 그리고 “tool_calls”에 해당 하는 값을 꺼내 tool_calls 변수에 담습니다. 확인을 위해 print 문을 사용하여 결과를 터미널 창에 출력합니다. 이를 통해 반환된 값이 제대로 형성되었는지 확인할 수 있습니다.

조각난 정보를 모아 딕셔너리 형태로 정리하기(2)

■ stock_info_streamlit.py

```

(... 생략 ...)
print('\n=====')
print(content)

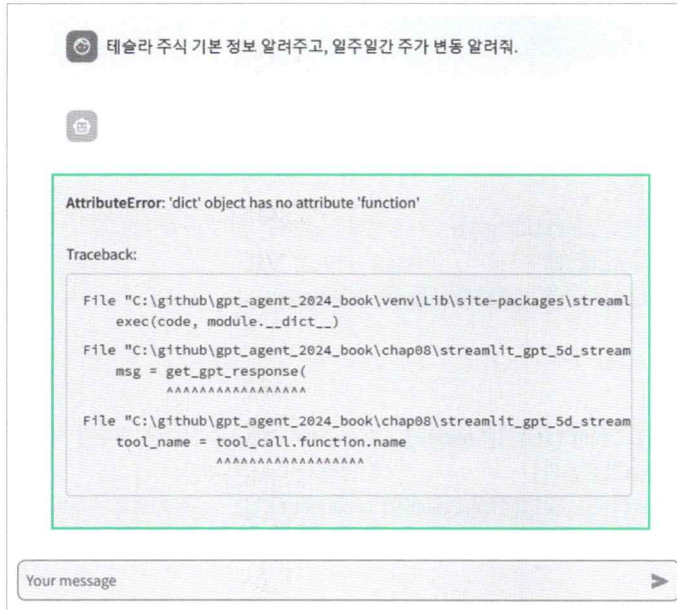
# print('\n===== tool_calls_chunk')
# for tool_call_chunk in tool_calls_chunk:
#     print(tool_call_chunk)
tool_obj = tool_list_to_tool_obj(tool_calls_chunk)
tool_calls = tool_obj["tool_calls"]
print(tool_calls)

if tool_calls: # tool_calls가 있는 경우
    ...

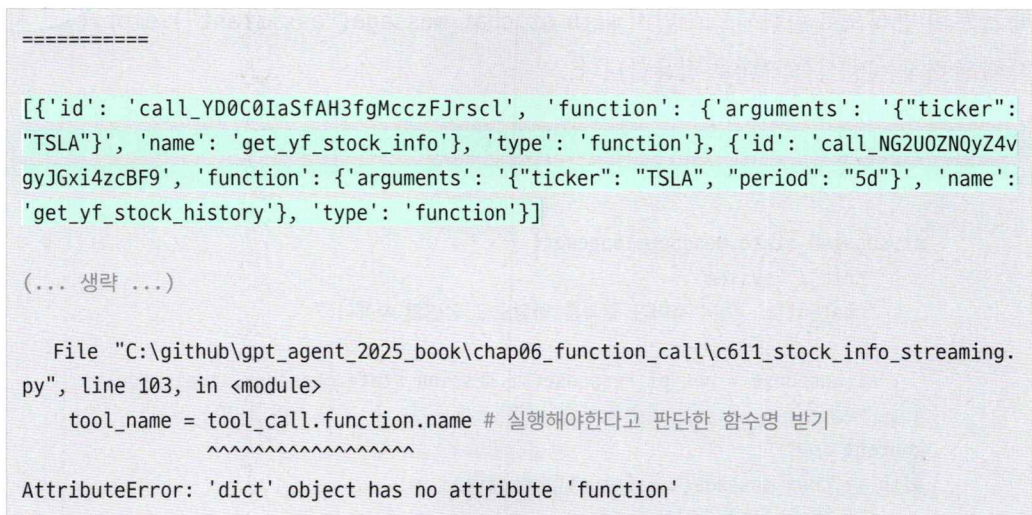
(... 생략 ...)

```

스트림릿을 실행하고 ‘테슬라 주식 기본 정보 알려주고 일주일간 주가 변동도 알려줘’라고 질문했습니다. Stream이 True로 설정된 상태에서 이를 처리하는 코드가 아직 제대로 만들지 않았으므로 스트림릿이 돌아가는 웹 브라우저에서는 오류 메시지가 나옵니다.



살펴봐야 할 것은 터미널 창에 출력된 메시지입니다. 조각나 있던 평선 콜링 관련 정보들이 다음처럼 깔끔하게 결합되어 있습니다.



5. 이제 `tool_calls`가 있을 때 함수를 실행할 수 있도록 코드를 수정합니다. `tool_list_to_tool_obj` 함수를 이용해 `tool_calls`를 딕셔너리 형태로 변환했으므로 딕셔너리 형태로 읽어 올 수 있도록 수정합니다.

```

딕셔너리 tool_call 값 읽어 오기(1) stock_info_streamlit.py

(... 생략 ...)

if tool_calls: # tool_calls가 있는 경우
    for tool_call in tool_calls:
        # tool_name = tool_call.function.name # 실행해야 한다고 판단한 함수명 받기
        # tool_call_id = tool_call.id # 함수 아이디 받기
        # arguments = json.loads(tool_call.function.arguments)
# 문자열을 딕셔너리로 변환

        # 딕셔너리 형태에서 받기
        tool_name = tool_call["function"]["name"] # 실행해야 한다고 판단한 함수명 받기
        tool_call_id = tool_call["id"] # 함수 아이디 받기
        arguments = json.loads(tool_call["function"]["arguments"]) # 문자열을 딕셔너리
로 변환

        if tool_name == "get_current_time": # tool_name이 "get_current_time"인 경우
            (... 생략 ...)

```

6. 함수를 실행한 후, 그 결과를 다시 `get_ai_response` 함수를 이용해 받아 와야 합니다. 이 때 스트림 방식으로 출력되므로 앞서 `with st.chat_message("assistant").empty()`로 시작해 출력하는 방식을 그대로 적용합니다.

```

딕셔너리 tool_call 값 읽어 오기(2) stock_info_streamlit.py

(... 생략 ...)

st.session_state.messages.append({
    "role": "system",
    "content": "이제 주어진 결과를 바탕으로 답변할 차례다."
})

ai_response = get_ai_response(st.session_state.messages, tools=tools)
# ai_message = ai_response.choices[0].message # 주석 처리 하거나 삭제
content = ""
with st.chat_message("assistant").empty():
    for chunk in ai_response:
        content_chunk = chunk.choices[0].delta.content
        if content_chunk:

```

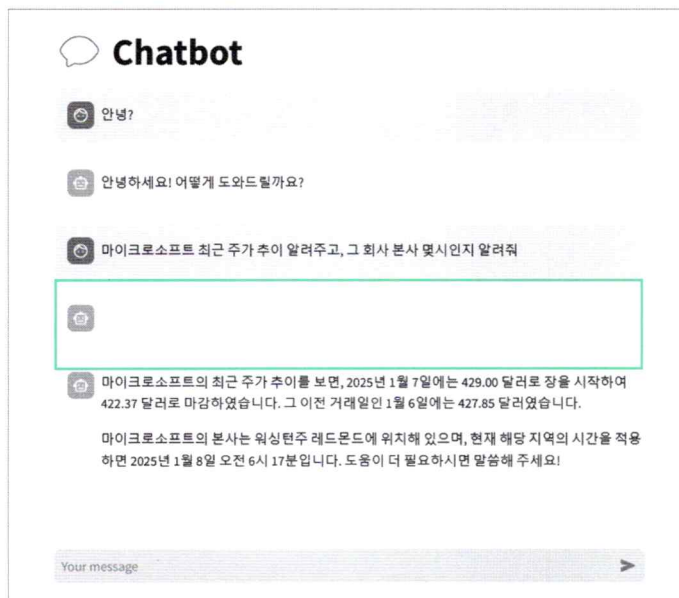
```

        print(content_chunk, end='')
        content += content_chunk
        st.markdown(content) # 스트림릿 챗 메시지에 markdown으로 출력
st.session_state.messages.append({
    "role": "assistant",
    "content": content
}) # AI 응답을 대화 기록에 추가

print("AI\t: " + content) # AI 응답을 출력

```

코드를 실행해 보면 다음처럼 잘 작동합니다. 그런데 대화 창에 빈 메시지 칸이 보입니다. 이는 처음 `get_ai_response` 함수를 실행할 때 평션 콜링이 필요한 경우에도 빈 메시지 칸을 생성하는데 그 칸에 `content`가 없어서 공백으로 남기 때문입니다. 이 부분은 어떤 함수를 임시로 사용했는지 보여 주는 용도로 활용해 보겠습니다.



7. `tool_obj`, `tool_calls`를 처리하던 코드를 `with st.chat_message("assistant").empty()` 안으로 옮깁니다. 그리고 `for` 문을 사용하여 `st.write`로 `tool_calls` 내용 중에서 `function`에 해당하는 부분만 모아서 출력하도록 수정합니다.

```
(... 생략 ...)
with st.chat_message("assistant").empty(): # 스트림릿 챗 메시지 초기화
    (... 생략 ...)
    # print(chunk) # 임시로 chunk 출력
    if chunk.choices[0].delta.tool_calls: # tool_calls가 있는 경우
        tool_calls_chunk += chunk.choices[0].delta.tool_calls # tool_calls_
chunk에 추가

    tool_obj = tool_list_to_tool_obj(tool_calls_chunk) # 아래에서 여기로 이동
    tool_calls = tool_obj["tool_calls"] # 아래에서 여기로 이동

    if len(tool_calls) > 0: # 만약 tool_calls가 존재하면, st.write로 tool_call 내용 출력
        print(tool_calls)
        # tool_calls에서 function 정보만 모아서 출력
        tool_call_msg = [tool_call["function"] for tool_call in tool_calls]
        st.write(tool_call_msg)
    print('\n=====')
    print(content)
    # tool_obj, tool_calls 있던 위치인데 위로 옮김

    if tool_calls: # tool_calls가 있는 경우
    (... 생략 ...)
```

이 코드를 실행하면 빈 상태로 남아 있던 칸에 GPT가 실행할 함수의 이름과 인자값이 출력됩니다.

